

How to Avoid Priority Inversion and Enable Priority Inheritance in Linux Kernel Programming

This article focuses on avoiding priority inversion with the help of priority inheritance in real-time process execution, which accesses the kernel code. Kernel synchronisation mechanisms such as semaphore and rt-mutex are also covered. Priority inheritance in rt-mutex eliminates the priority inversion problem and hence improves real-time performance in the Linux environment.



Application programs access hardware through the kernel code. Improper synchronisation or locking at the kernel can result in random crashes and unexpected results. To resolve this issue, multiple kernel level synchronisation and locking mechanisms have been developed. This article mainly focuses on semaphore and rt-mutex.

Semaphore: This is a synchronisation mechanism that implements mutual exclusion with a variable and associated library functions. This variable is used to solve critical section problems and to achieve process synchronisation in a multi-processing environment.

rt-mutex: This extends the semantics of simple mutexes by the priority inheritance protocol. A low priority owner of

rt-mutex inherits the priority of a higher priority waiter until rt-mutex is released. If the temporarily boosted owner blocks rt-mutex, it propagates the priority boosting to the owner of the other rt-mutex on which it is blocked. Priority boosting is immediately removed once rt-mutex has been unlocked.

The priority inversion problem

Let's consider three processes with high, middle and low priorities.

Tasks

High priority (H)

Medium priority (M)

Low priority (L)